



HABIB UNIVERSITY

Data Structures & Algorithms

CS/CE 102/171 Spring 2025

Instructor: Maria Samad

Quiz # 2 Solution

QUESTION 1: ALGORITHM DESIGN USING PRIORITY QUEUES (10 marks)

- **Ticket Management with Priority Queues:**
 - After the phenomenal success of the movie, “The Legend of Maula Jatt”, it is finally going to be screened in different multiplexes all across India. A multiplex contains multiple types of theaters, represented by upper-case alphabets, i.e. ‘A’, ‘B’, ‘C’, etc, which also represent the priorities in alphabetical ascending order. The movie will be screened in all types of theaters.
 - For a specific show in each theater, people can line up to purchase tickets, which is recorded as an array of tuples by the administration, in which each tuple is defined as (“Theater Type”, Number of tickets purchased). This purchases array is read-only, which means nothing can be modified in it by your algorithm.
 - This purchases array contains ‘n’ number of elements, where $n > 1$. Assume n is large enough to fit all the purchases. Both n and purchases array are given in the correct format, so no need to do error checking or taking input, and must be used directly. However, each of them should be declared/defined in the algorithm.
 - Another array called ***theaters***, which is an array of theater types is also provided. Don’t forget to declare it though
 - Once all the purchases have been made, the multiplex owner wants to share the purchases in order of their priority with the movie distributors. Assume there is at least 1 ticket purchased for each theater type, which means no entry in priority queue will be of type (‘A’, 0) or there won’t be any missing theater entries in the array.
 - However, the list is too long and too confusing to be read, so they ask their team to combine all the tickets purchases of each theater, and represent them as a sorted Priority Queue, such that the Priority Queue will contain a tuple with theater type as first element, and tickets purchased for that theater as a regular queue nested in the tuple as its second element. This queue will be of exact size as required, without any additional None values.
 - Priority Queue has a fixed size, which means its size depends on the number of types of theaters. If there are 4 types of theaters, it will have 4 entries exactly, and not more nor less. This means, the priority queue should be created of fixed size according to the theater types. Initially it contains None values, however, once the process is complete, the resultant Priority Queue will contain the actual elements.
 - **For example:** The array of tickets, purchases = [(‘A’, 6), (‘B’, 4), (‘C’, 8), (‘A’, 11), (‘A’, 5), (‘C’, 2), (‘B’, 11), (‘B’, 1), (‘B’, 14), (‘C’, 17), (‘A’, 3), (‘C’, 7), (‘A’, 10), (‘C’, 10), (‘C’, 15), (‘B’, 12), (‘C’, 9), (‘B’, 10), (‘C’, 5), (‘B’, 3), (‘C’, 6)]
 - There are 3 types of theaters, as seen from the array above, which are ‘A’, ‘B’ and ‘C’. Hence initially, a Priority Queue is generated of size 3 containing None values, i.e., something like: PQ = [None, None, None]
 - Then resulting priority queue would be: PQ = [(‘A’, [6, 11, 5, 3, 10]), (‘B’, [4, 11, 1, 14, 12, 10, 3]), (‘C’, [8, 2, 17, 7, 10, 15, 9, 5, 6])]
 - Design an algorithm to implement the ticket management task as explained above.
- **REQUIREMENTS:**
 - Do not **HARD CODE** your design for the given example. The algorithm should work for any valid sized arrays
 - You can assume enqueue and dequeue functions of Priority Queues are given to behave like a regular queue without taking into consideration about the sorted order, which means your algorithm must design that functionality to add to Priority Queues in sorted fashion by using the enqueue and dequeue functions provided.
 - Declare all the data structures and variables being used in this algorithm, including any inputs, if any.
 - The “purchases” and “theaters” arrays should only be used for reading purposes **ONLY**.
 - The algorithm should be able to add entries to the resultant Priority Queues in an ascending sorted manner.
 - **DO NOT USE ANY OTHER DATA STRUCTURES INCLUDING List/ADT/Python lists apart from the ones permitted to use**
 - **REMEMBER:** Priority Queues are non-iterable and non-traversing data structures, so you CANNOT and MUST NOT do indexing within the data structure to access the elements.

SOLUTION:

- n – number of entries in purchases array
- purchases – array of tuples containing n elements
- theaters – array of theater types
- n_theaters – an integer that represents the number of entries in theaters array
- PQ – resultant priority queue of size n_theaters, initially filled with None values

#Helper function to add to PQ in ascending sorted order

- Function header: addToPQ(PQ, i, nestedQ) where PQ is the priority queue in which information needs to be added; i is the theater type; nestedQ is the queue in the resultant PQ
 - size = size of the PQ passed as argument in the function
 - PQtuple – forming a tuple to be inserted into PQ = (i, nestedQ)
 - count – an integer variable

#Logic for adding to PQ in ascending sorted manner

- if PQ is empty, then simply add the PQtuple directly and it will be considered sorted as there is only one entry
- Otherwise, do the following:
 - PQelement = PQ.dequeue()
 - count = size
 - while (PQelement[0] < i), do the following:
 - PQ.enqueue (PQelement)
 - PQelement = PQ.dequeue()
 - count = count – 1
 - PQ.enqueue(PQtuple)
 - for count – 1 times, do the following:
 - PQ.enqueue (PQelement)
 - PQelement = PQ.dequeue()
 - PQ.enqueue (PQelement)

#Now that we have the helper function, let's call it to solve the given problem

- for i in theaters, do as follows:
 - Qsize – an integer to calculate the exact size of nested queue required for the resultant PQ, initialize it to 0

#calculate the exact size of queue from the purchases array

- For n times, do the following
 - If purchases[j][0] == i, increase Qsize by 1
 - Otherwise, just continue with the loop

#now that we know the exact size, we can create the nested queue

- nestedQ – a regular queue of size Qsize, initially empty
- for n times, do as follows:
 - if purchases[j][0] == i, then:
 - nestedQ.enqueue(purchases[j][1])

#Once we have the queue set up, use the helper function to add the elements to the PQ

- addToPQ(PQ, i, nestedQ)

QUESTION 2: BINARY SEARCH (10 marks)

Your supervisor gives you the following code for an algorithm using Binary Search, but it is full of errors. For the given program, find the problems and specify their solutions. Use the empty column to specify the Line No. where the error might be present, state the problem in that line, and then a quick solution to the problem. You may state the solution in words, as well, if you are not comfortable with specifying the solution in terms of Python code. For algorithm description about what it is supposed to do, read the description after the program. Please remember, there are no indentation errors in the program, so don't try to specify wrong indentation as one of the faults in this program

Line No.	def find_k_nearest_neighbors(arr, k, target):	
1	neighbors_list = []	
	# Binary search to find the index of the closest	
2	# element to the target	
3	left, right = 0, len(arr)	
4	while left <= right:	Line 4: Wrong condition for Binary Search loop
5	mid = (left + right) // 3	Line 5: Should be dividing by 2 as per the Binary Search algo
6	if arr[mid] == target:	
7	left = mid - 1	Line 7: When target in middle, you end it, not move to the left
8	break	
9	elif arr[mid] > target:	
10	left = mid + 1	Line 10: Wrong update of left, instead of right
11	else:	
12	right = mid - 1	Line 12: Wrong update of right, instead of left
13	closest_index = right	Line 13: Incorrect variable chosen for final assignment/selection
14		
15	if closest_index <= 0:	
16	neighbors_list = arr[0:k-1]	Line 16: Incorrect slicing causes the edge cases to fail
17	elif closest_index >= len(arr):	
18	neighbors_list = arr[len(arr)-k:len(arr)-1]	Line 18: Incorrect slicing causing the edge case to fail
19	else:	
20	neighbor_count = 1	Line 20: Incorrect initialization
21	if arr[closest_index] != target:	
22	neighbors_list.append(arr[closest_index])	
23	neighbor_count += 2	Line 23: Should increment by 1
24		
25	closest_value = target	
26		
27	for i in range(1, k - 1):	Line 27: Incorrect loop range, must be k times
28	if (closest_index - i) > 0:	Line 28: The condition for 0 is missing so it should be >=
29	diff = arr[closest_index - i] - closest_value	Line 29: Absolute difference missing
30	if diff >= k:	Line 30: Incorrect condition
31	neighbor_count += 1	
32	neighbors_list.append(arr[closest_index - i])	Line 32: It should be inserting in the beginning
33	if neighbor_count >= k:	Line 33: Incorrect condition
34	break	
35	if (closest_index + i) < (len(arr) + 1):	Line 35: Incorrect length – it's off by 1
36	diff = abs(arr[closest_index + i] - closest_value)	
37	if diff < k:	Line 37: Incorrect condition
38	neighbor_count += 1	
39	neighbors_list.insert(0, arr[closest_index + i])	Line 39: It should be appending to the end
40	if neighbor_count >= k:	
41	break	
42		
43	return neighbor_list	Line 43: Incorrect variable name
44		
45	# Example usage:	
46	arr = [1, 3, 5, 6, 8, 10, 12]	
47	k = 4	
48	target = 6	
49	print(find_k_nearest_neighbors(arr, k, target))	

Algorithm Description of the Binary Search Program given on previous page.

We are using binary search to achieve the k nearest neighbor of the target in an integer array, as explained below:

- Given a sorted integer array, find the k closest elements with a difference $\leq k$ to the target value in the list where k and target are positive integers, taken as input from the user. The target may or may not be present in the input array. If the target is less than or equal to the first element in the input array, return first k elements. Similarly, if the target is more than or equal to the last element in the input array, return the last k elements. If the target is somewhere in between, then the k elements closest to the target value should be returned. The returned elements should be in the same order as present in the input array.
- For example,
 - Input: [10, 12, 15, 17, 18, 20, 25], k = 4, target = 16 → Output: [12, 15, 17, 18]
 - Input: [2, 3, 4, 5, 6, 7], k = 3, target = 1 → Output: [2, 3, 4]
 - Input: [2, 3, 4, 5, 6, 7], k = 2, target = 8 → Output: [6, 7]
- The output is in the form of a Python list

QUESTION 3: INSERTION SORT (10 marks)

Sort the given list in descending order using insertion sort algorithm.

- [43, 92, 31, 14, 27, 65, 79, 16, 58, 90, 2]

SHOW ALL INTERIM CALCULATIONS IN ORDER TO GET COMPLETE/FULL MARKS. An answer without the intermediate steps will directly result in 0 marks.

SOLUTION: On next page

~~Date:~~ Q3 Insertion Sort

[43, 92, 31, 14, 27, 65, 79, 16, 58, 90, 2]

[43, 92, 31, 14, 27, 65, 79, 16, 58, 90, 2]
sorted unsorted

[92, 43, 31, 14, 27, 65, 79, 16, 58, 90, 2]
sorted unsorted

[92, 43, 31, 14, 27, 65, 79, 16, 58, 90, 2]
sorted unsorted

[92, 43, 31, 14, 27, 65, 79, 16, 58, 90, 2]
sorted unsorted

[92, 43, 31, 27, 14, 65, 79, 16, 58, 90, 2]
sorted unsorted

[92, 65, 43, 31, 27, 14, 79, 16, 58, 90, 2]
sorted unsorted

[92, 79, 65, 43, 31, 27, 14, 16, 58, 90, 2]
sorted unsorted

[92, 79, 65, 43, 31, 27, 16, 14, 58, 90, 2]
sorted unsorted

[92, 79, 65, 58, 43, 31, 27, 16, 14, 90, 2]
sorted unsorted

[92, 90, 79, 65, 58, 43, 31, 27, 16, 14, 2]
sorted unsorted

DALMATIAN

[92, 90, 79, 65, 58, 43, 31, 27, 16, 14, 2]

SORTED!